

CSA1715-ARTIFICIAL INTELLIGENCE

LAB PROGRAM- 10

30-07-2026

10. Write the python program to implement A* algorithm.

Aim

To implement the A* Search Algorithm in Python to find the shortest path from a start node to a goal node using heuristic values.

Algorithm

1. Start the program.
2. Define the graph and the cost between connected nodes.
3. Define the heuristic value of each node.
4. Select the starting node and goal node.
5. Set the initial path cost g of the starting node to zero.
6. Calculate the evaluation function using
$$f(n) = g(n) + h(n).$$
7. Select the node having the lowest f(n) value.
8. Examine its neighbouring nodes.
9. Calculate the new path cost for each neighbour.
10. Update the path if a shorter cost is found.
11. Continue the process until the goal node is reached.
12. Display the shortest path and its total cost.
13. Generate the graph using NetworkX and highlight the shortest path.
14. Stop the program.

Program

```
import networkx as nx  
  
import matplotlib.pyplot as plt  
  
import heapq
```

```
g = {
```

```
'A': [('B',1),('C',4)],  
'B': [('D',2),('E',5)],  
'C': [('E',1)],  
'D': [('G',3)],  
'E': [('G',2)],  
'G': []  
}
```

```
h = {'A':6,'B':4,'C':3,'D':2,'E':1,'G':0}
```

```
start, goal = 'A', 'G'
```

```
pq = [(h[start], 0, start, [start])]
```

```
visited = set()
```

```
while pq:
```

```
    f, cost, node, path = heapq.heappop(pq)
```

```
    if node in visited:
```

```
        continue
```

```
    visited.add(node)
```

```
    if node == goal:
```

```
        best_path, minimum = path, cost
```

```
        break
```

```
    for nxt, w in g[node]:
```

```
        if nxt not in visited:
```

```

        new_cost = cost + w
        heapq.heappush(
            pq,
            (new_cost + h[nxt], new_cost,
             nxt, path + [nxt])
        )

print("Shortest Path:", " -> ".join(best_path))
print("Minimum Cost:", minimum)

G = nx.Graph()

for node in g:
    for nxt, w in g[node]:
        G.add_edge(node, nxt, weight=w)

pos = {
    'A':(0,1),
    'B':(1,2),
    'C':(1,0),
    'D':(2,2),
    'E':(2,0),
    'G':(3,1)
}

nx.draw(G, pos, with_labels=True,
        node_size=1800, node_color="lightblue")

```

```
nx.draw_networkx_edge_labels(  
    G, pos, nx.get_edge_attributes(G,'weight'))  
  
edges = [(best_path[i], best_path[i+1])  
         for i in range(len(best_path)-1)]  
  
nx.draw_networkx_edges(  
    G, pos, edgelist=edges, width=4)  
  
plt.title("A* Algorithm - Shortest Path")  
plt.show()
```

Input

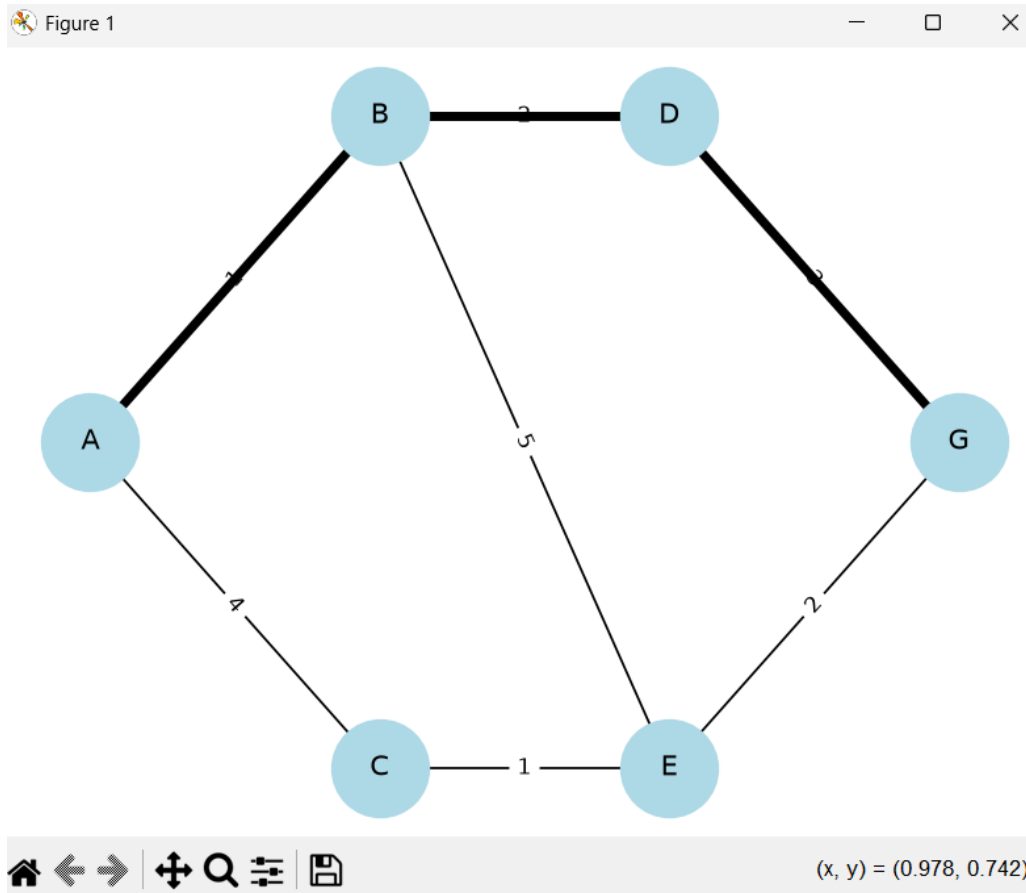
Start Node: A

Goal Node: G

Output

Shortest Path: A -> B -> D -> G

Minimum Cost: 6



Result:

The A* Search Algorithm was implemented successfully. The program used the evaluation function $f(n)=g(n)+h(n)$ to determine the optimal path from the start node to the goal node with the minimum cost.